Math 105LA Assignment 2

Fall 2018

Due Sun 10/13, 11pm

Write a MATLAB function to implement the Newton's method:

```
Input: f, df, p0, N, tol, tol_fp
Output: p
Step 1: Set n = 1, q = f(p0)
Step 2: While n <= N, repeat step 3 to step 6
Step 3:     Set p = p0 - q/df(p0), q = f(p)
Step 4:     If |q| < tol_fp
                  Output('f(c) is smaller than tol_fp', p), stop
            elseif |p-p0| < tol
                  Output('p is converged with |p_{n+1}-p_{n}| smaller than tol', p) stop
            end
Step 5:     Set n = n+1
Step 6:     Set p0 = p
Step 7: Output('Maximum number of iteration is reached' ,p)
```

You can start with the template below. Make sure that you use the **same order** of input:

```
function p = newton(f,df,p0,N,tol,tol_fp)
% f = the function to be solved
% df = the derivative of f
% p0 = initial guess
% tol = tolerance for |p-p_0|
% tol_fp = tolerance for |f(p)|
% N = maximum number of iterations

%%%% type your code below  %%%%
% ……
%     ……
%           fprintf('|f(p)| is smaller than %0.2e', tol_fp)
%           return
%     ……
%           fprintf('|f(p)| is smaller than %0.2e', tol_fp)
%           return
%     ……
% ……
% disp('Maximum number of iteration is reached')
end
```

Note: in Step 4, there is no need to output p since it's already being returned. In Step 3, we need to know $f'(x)$. Taking derivatives in MATLAB requires the Symbolic Math Toolbox, see https://www.mathworks.com/help/symbolic/differentiation.html. Here, we do a bit extra work and provide the derivative to the algorithm.

For example, suppose that we would like to find a root of the equation $e^x - 3x = 0$ between 0 and 1 with $|p_{n+1} - p_n|$ smaller 10^{-5} or $|f(p_n)|$ smaller 10^{-6}, using at most 300 iterations. First, define the function and its derivative:

```
f = @(x) exp(x)-3*x;
fp = @(x) exp(x)-3; % f'(x)
```

Then, assuming your function is saved in '`newton.m`', type the following in the command window (without the semicolon):

```
newton(f,fp,0,300,1e-5,1e-6)
```

And your function should return `0.6191`.

More examples:

Define …	And it should output …

`f = @(x)1/(x-1)/(2-x)-6*exp(-1e+2*(x-1.5)^2);`
`df=@(x)-1/(x-1)^2/(2-x)+1/(x-1)/(2-x)^2+2*6*1e+2*(x-1.5)*exp(-1e+2*(x-1.5)^2);`
`p0 = 1.00001; N = 100; tol = 1e-4; tol_fp = 1e-4;`
`p = newton(f,df,p0,N,tol,tol_fp)`
 	
`p0 = 1.00001; N = 100; tol = 1e-8; tol_fp = 1e-4;`
`p = newton(f,df,p0,N,tol,tol_fp)`
 	
`p0 = 1.001; N = 100; tol = 1e-4; tol_fp = 1e-4;`
`p = newton(f,df,p0,N,tol,tol_fp)`
	`p is converged with |p_{n+1}-p_{n}| smaller than 1.00e-04`
	`p =`
	
	`1.0000`
	
	`|f(p)| is smaller than 1.00e-04`
	`p =`
	
	`1.4376`
	
	`|f(p)| is smaller than 1.00e-04`
	`p =`
	
	`1.4376`

`f = @(x) x^2 - 3;`
`df=@(x) 2*x;`
`p0 = 1; N = 100; tol = 1e-4; tol_fp = 1e-4;`
`p= newton(f,df,p0,N,tol,tol_fp)`
	`|f(p)| is smaller than 1.00e-04`
	`p =`
	
	`1.7321`

`f = @(x)5*exp(x)-4*x+3*x^2-x^3-10;`
`df=@(x) 5*exp(x)-4+6*x-3*x^2;`
`p0 = 0; N = 100; tol = 1e-4; tol_fp = 1e-8;`
`p = newton(f,df,p0,N,tol,tol_fp)`
	`p is converged with |p_{n+1}-p_{n}| smaller than 1.00e-04`
	`p =`
	
	`0.8638`

Submit a single file named by your UCInetID (e.g. `wtleung.m`) that contains the function. Your code will be tested on a few problems. Your grade of this assignment is calculated by the number of problems that your code solves correctly. Make sure your function runs without errors and has the same order of input above. **Failure to do so will result in a zero grade.**